

LLMOps & Prompt Versioning

Giảng viên · VinUniversity · Ngày 22

HÃY SUY NGHĨ...

"Prompt thay đổi = behavior thay đổi."

Case Study

Một team thay đổi 1 dòng trong system prompt để output "chi tiết hơn một chút".

Kết quả:

- Latency tăng 3x vì output dài hơn
- Token cost tăng 200%
- Không ai biết nguyên nhân — vì không track prompt versions

LLMOps giúp bạn: trace · version · eval mọi thay đổi.

Giữ câu hỏi này trong đầu suốt buổi học hôm nay

01

LLMOps vs MLOps Truyền Thống

20 phút

03

Prompt Hub: Version Control

30 phút

05

W&B Weave: LLM Evaluation Platform

15 phút

07

Live Demo & Thực Hành Labs

Còn lại

02

LangSmith: Trace, Debug & Monitor

40 phút

04

LLM Evaluation: Vượt Ra Ngoài Accuracy

20 phút

06

Guardrails & Safety Monitoring

15 phút

08

Key Takeaways & Preview Ngày 23

5 phút

Sau buổi học này, bạn sẽ có thể:

01

Master LangSmith Tracing

Instrument LLM apps, trace từng call, debug bottlenecks, filter traces theo latency/cost/error.

02

Implement Prompt Versioning

Dùng LangSmith Prompt Hub để store, version, pin prompts — giống GitHub cho prompts.

03

Setup LLM Evaluation hệ thống

Dùng W&B Weave + RAGAS để evaluate faithfulness, relevance, hallucination có hệ thống.

04

Áp dụng Guardrails an toàn

Validate LLM inputs/outputs: chặn PII, detect prompt injection, reask khi format sai.

Những gì bạn phải nộp sau buổi thực hành:

1. LangSmith Project

> 100 traces từ RAG pipeline thực tế. Phân tích được latency, cost, error rate.

2. Prompt Hub

2 prompt versions với commit messages rõ ràng. Implement A/B routing 50/50.

3. RAGAS Evaluation Report

Chạy RAGAS trên 50 QA pairs. Faithfulness score > 0.8. So sánh 2 prompt versions.

4. Guardrails AI Validator

Block PII (email, phone). Auto-reformat non-JSON outputs. Log mọi incident.

PHẦN 01

LLMOps vs MLOps Truyền Thống

Tại sao vận hành LLM lại khác?

MLOps (Machine Learning Operations) là quy trình vận hành các hệ thống ML truyền thống:

Tracking	Versioning	Evaluation	Safety
• Hyperparameters (lr, batch size) • Train/eval metrics (loss, accuracy) • Model artifacts & checkpoints • Data versions	• Model weights (.pkl, .pt) • Training datasets • Feature pipelines • Code (Git)	• Accuracy, F1, AUC-ROC • Precision / Recall • Confusion matrix • Benchmark datasets	• Model fairness metrics • Bias detection • Data privacy (training) • Adversarial robustness

LLMOps extends MLOps với các thách thức đặc thù của Large Language Models:

Non-deterministic

Cùng 1 input → output khác nhau mỗi lần.
Không thể reproduce bug dễ dàng.

Prompt-sensitive

Thay 1 từ trong prompt → behavior thay đổi hoàn toàn. Phải version control prompts.

Subjective quality

Không có ground truth rõ ràng. Cần LLM-as-judge hoặc human eval.

Token cost

Cost tính theo token, không phải compute time. 1 prompt dài = nhiều tiền hơn.

Safety concerns

Prompt injection, jailbreak, PII leak — không tồn tại trong MLOps truyền thống.

Hallucination

LLM có thể tự bịa ra thông tin. Phải track hallucination rate liên tục.

MLOps vs LLMOps — So Sánh Chi Tiết

Section 01

Khía cạnh	MLOps Truyền Thống	LLMOps
Tracking	Hyperparams, train/eval metrics	Trace từng LLM call, token cost per request
Output	Deterministic, reproducible	Non-deterministic, subjective quality
Versioning	Model weights, datasets, code	+ Prompts, system instructions
Evaluation	Accuracy, F1, AUC-ROC	Faithfulness, Relevance, Hallucination rate
Cost model	Compute time (GPU hours)	Token cost per task
Safety	Model fairness, bias	Prompt injection, jailbreak, PII leak
Key tools	MLflow, W&B, DVC	LangSmith, W&B Weave, Helicone, RAGAS

Safety Layer

Guardrails AI · Llama Guard 2 · NeMo Guardrails

Block harmful inputs/outputs, detect PII, classify harm categories

Evaluation Layer

RAGAS · Promptfoo · DeepEval · LLM-as-Judge

Measure faithfulness, relevance, hallucination; A/B test prompts

Tracing Layer

LangSmith · W&B Weave · Phoenix · Arize

Trace every LLM call, debug errors, analyze latency & cost per run

Prompt Management

LangSmith Prompt Hub · YAML in Git · Promptsmith

Version, store, review, pin exact prompt versions in production

Cost & Observability

Helicone · OpenMeter · Prometheus · Grafana

Track token cost, request volume, latency P50/P95, budget alerts

Key Metrics Cần Track Trong LLM Ops

Section 01

\$0.002

avg per query

Token Cost / Task

< 5%

threshold

Hallucination Rate

> 0.8

RAGAS target

Faithfulness Score

< 3s

SLA target

Latency P95

Metrics bổ sung cho LLM hệ thống:

- User satisfaction score (thumbs up/down, CSAT)
- Context precision & recall (RAG quality)
- Prompt injection attempt rate
- Cache hit rate (reduce redundant LLM calls)
- Error rate by error type (timeout, rate limit, invalid output)
- Model drift: so sánh output quality theo thời gian

PHẦN 02

LangSmith

Trace · Debug · Monitor LLM Applications

LangSmith là observability platform được xây dựng riêng cho LLM applications:

Tracing

Ghi lại toàn bộ execution tree của LLM app: root run → child runs (LLM calls, tool calls, retrieval). Xem latency breakdown từng bước.

Monitoring

Dashboard theo dõi latency heatmap, cost per day/user, error rate trend, token usage breakdown theo thời gian thực.

Datasets

Annotate traces → tạo evaluation dataset tự động. Dataset grows organically từ production traffic.

Prompt Hub

Store, version, pull/push prompts. Pin exact version bằng commit hash. A/B test prompt versions.

Evaluations

Chạy evaluations tự động trên datasets. So sánh kết quả qua các deployments để detect regressions.

Human Review

Annotation Queues: human reviewers label outputs. Xây dựng golden datasets cho calibration.

```
import os

# 1. Enable tracing
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "ls_..."
os.environ["LANGCHAIN_PROJECT"] = "rag-prod"

# 2. Auto-instrument LangChain (zero code change)
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model="gpt-4o-mini")
prompt = ChatPromptTemplate.from_template("...")
chain = prompt | llm

# 3. Custom spans for non-LangChain code
from langsmith import traceable

@traceable(run_type="chain", name="my_rag_pipeline")
def rag_pipeline(question: str) -> str:
    docs = retriever.invoke(question)
    context = format_docs(docs)
    return llm.invoke(f"Context: {context}\nQ: {question}")
```

Trace Anatomy

Root run → child runs (tree view)

LLM calls với full input/output

Tool calls: search, DB lookup

Retrieval: docs fetched, scores

Latency breakdown mỗi step

Filtering Traces

Error traces only

Latency > 2 seconds

Cost > \$0.01 per call

By model, tag, date range

Trace Anatomy — Hiểu Cấu Trúc Một Run

Section 02

```
ROOT: rag_pipeline
```

```
└ RETRIEVER: vectorstore.similarity_search [120ms] 3 docs
```

```
└ CHAIN: format_prompt [5ms]
```

```
└ LLM: gpt-4o-mini [1.4s] ← ⚠ bottleneck
```

```
└ input_tokens: 542 output_tokens: 198 cost: $0.0008
```

```
└ finish_reason: stop latency: 1412ms
```

Từ một trace này bạn biết ngay: bottleneck ở LLM call, retrieval nhanh, format nhẹ → tối ưu bằng cách giảm output token hoặc dùng streaming.

Quy Trình Tạo Dataset Từ Production

- 1 Production traces chạy liên tục
- 2 Annotate traces quan trọng (đúng/sai, edge cases)
- 3 Add to evaluation dataset (1-click)
- 4 Dataset grows organically từ production
- 5 Run regression test mỗi lần deploy mới

Dashboard Metrics

Latency heatmap

Xem latency theo time window, identify peak hours

Cost per day/user

Track budget, set alerts khi vượt threshold

Error rate trend

% failed runs theo ngày, breakdown by error type

Token usage

Input vs output tokens, model distribution

Custom A/B tags

Tag mỗi trace với experiment version để so sánh

PHẦN 03

Prompt Hub

Version Control cho Prompts — Tại Sao Quan Trọng?

Tại Sao Phải Version Control Prompts?

Section 03

Prompt là code. Nếu bạn không track changes, bạn đang vận hành trong bóng tối.

Vấn Đề Khi Không Version Control

- × Prompt bị edit trực tiếp → không biết ai thay đổi gì
- × Cost tăng 200% → không tìm ra nguyên nhân
- × Latency tăng 3x → không rollback được
- × A/B test không chính xác → 2 versions bị trộn lẫn
- × Onboarding mới → không biết prompt nào đang dùng
- × Bug production → không reproduce được

Với Prompt Versioning

- ✓ Mỗi thay đổi có commit message rõ ràng
- ✓ Pin exact version bằng hash trong production
- ✓ Rollback 1-click khi có incident
- ✓ A/B test chính xác với isolated versions
- ✓ Team review prompt changes như review code
- ✓ Full audit trail: ai, khi nào, thay gì

```
from langchain import hub

# — PULL —
# Pull latest version
prompt = hub.pull("my-org/rag-system-prompt")

# Pin exact version using commit hash (PRODUCTION BEST PRACTICE)
prompt = hub.pull("my-org/rag-system-prompt:abc123de")

# Use the prompt in a chain
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"context": docs, "question": q})

# — PUSH —
from langchain.prompts import ChatPromptTemplate

new_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Language: {language}. \n\nContext: \n{context}"),
    ("human", "{question}")
])

hub.push(
    "my-org/rag-system-prompt",
    new_prompt,
    new_commit_message="feat: Add language param, tighten answer length"
)
```

Best Practices

Pin hash trong prod

Dùng :hash để lock version, không bị surprise update

Commit messages rõ

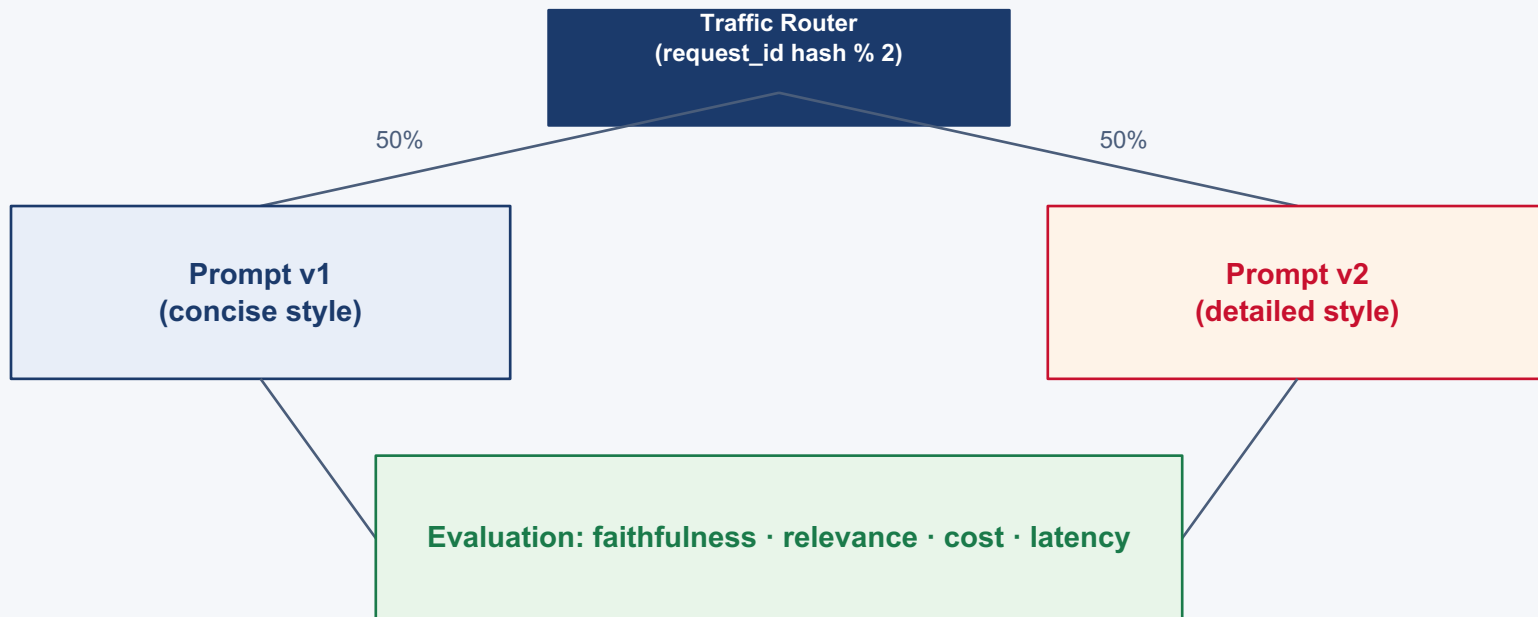
feat/fix/refactor prefix như Git conventional commits

Typed variables

Dùng {context}, {question} — để validate input

Team review

PR process: 1 người push, 1 người review trước deploy



Best practice: Thu thập > 100 queries trước khi kết luận. Tag mỗi trace với prompt version trong LangSmith để so sánh chính xác.

Nếu team dùng monorepo hoặc không muốn dùng Prompt Hub:

```
# prompts/rag-system-prompt.yaml
name: rag-system-prompt
version: "1.3.0"
description: "RAG system prompt with language support"
variables:
  - context
  - question
  - language

template: |
  You are a helpful assistant that answers questions based on
  the provided context. Always respond in {language}.

  Context:
  {context}

  Rules:
  - Answer ONLY based on the context above
  - If answer not in context, say "I don't know"
  - Keep answers under 200 words

  Question: {question}
```

Workflow với Git

- Prompts stored trong /prompts/*.yaml
- Mọi thay đổi qua Pull Request
- CI tự động test prompt khi có changes
- Load prompt từ YAML thay vì hardcoded
- Diff rõ ràng trong GitHub/GitLab
- Phù hợp cho team không muốn thêm tool

Khi nào dùng Prompt Hub vs Git?

Prompt Hub: team lớn, cần UI review, A/B test tự động.

Git YAML: dev-centric, monorepo, CI/CD tight integration.

PHẦN 04

LLM Evaluation

Vượt Ra Ngoài Accuracy — Đo Lường Chất Lượng Thực Sự

Tại Sao Accuracy Không Đủ Cho LLM?

Section 04

Ví dụ: model có accuracy 95% nhưng vẫn gây hại nếu 5% còn lại là hallucination trong context y tế.

Hallucination

Model trả lời sai nhưng nghe rất thuyết phục. Accuracy không phát hiện được vì không có ground truth.

Off-topic answers

Model trả lời đúng ngữ pháp, coherent — nhưng không liên quan câu hỏi. Accuracy bỏ qua.

Context mismatch

Trong RAG: model ignore retrieved context, dùng parametric knowledge → bịa ra từ training data.

Format non-compliance

Model không trả JSON khi được yêu cầu. Accuracy đo text match — miss hoàn toàn format errors.

Metric	Đo Gì?	Cách Tính	Target
Faithfulness	Output có dựa trên context không? (Hallucination detection)	% statements trong answer có thể verify từ context	> 0.8
Answer Relevance	Câu trả lời có đúng với câu hỏi không?	Similarity giữa câu hỏi và câu trả lời (reverse gen)	> 0.75
Context Precision	Chunks retrieved có thực sự liên quan không?	% retrieved chunks có trong ideal answer	> 0.7
Context Recall	Đủ context để trả lời đúng không?	% facts trong ground truth có trong context	> 0.8

Dùng một LLM mạnh (GPT-5) để đánh giá output của model nhỏ hơn — scalable hơn human eval 10x.



Ưu Điểm

- ✓ Scalable: chạy hàng nghìn evals/ngày
- ✓ Cheap hơn human eval 10-50x
- ✓ Consistent: không bị fatigue
- ✓ Không cần ground truth (reference-free)
- ✓ G-Eval, MT-Bench frameworks sẵn có

Hạn Chế & Cách Xử Lý

- ! Bias: LLM judge có positional/verbosity bias
- ! Cần calibrate định kỳ với human judgments
- ! Đo Cohen's kappa giữa human & LLM judge
- ! Không dùng cùng model để judge chính nó
- ! Prompt judge cần rất rõ ràng về rubric

Best practice: dùng LLM-as-judge để scale, human eval để calibrate. Kết hợp cho kết quả tốt nhất.

1

LLM-as-Judge (Daily)

Chạy tự động toàn bộ
eval suite mỗi ngày.
Cost thấp, scale tốt.

2

Human Review (Weekly)

Sample 100-200 outputs.
Annotators dùng Label
Studio / Argilla.

3

Calibration (Monthly)

So sánh LLM judge vs
human. Đo Cohen's
kappa agreement.

4

Update Judge Prompt

Nếu kappa < 0.7:
adjust rubric hoặc
change judge model.

Tools cho Human Eval:

Label Studio (open source) · Argilla (NLP-focused) · Scale AI (enterprise) · Prolific (crowdsourcing)
Inter-annotator agreement (IAA) > 0.7 là healthy. Gold standard samples → calibrate LLM judge.

PHẦN 05

W&B Weave

LLM Evaluation Platform — Auto-tracking, Model Comparison, Cost Awareness

```
import weave
import openai

# 1. Init Weave - auto-tracks ALL function calls
weave.init("my-llm-project")

# 2. Any decorated function is traced automatically
@weave.op()
def generate_answer(question: str, context: str) -> str:
    response = openai.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": f"Context: {context}"},
            {"role": "user", "content": question}
        ]
    )
    return response.choices[0].message.content

# 3. Define Scorer (evaluation metric)
class FaithfulnessScorer(weave.Scorer):
    @weave.op()
    def score(self, output: str, context: str) -> dict:
        # Use LLM-as-judge
        verdict = llm_judge(output, context)
        return {"faithful": verdict["score"] > 0.7, "score":
verdict["score"]}
```

Tính Năng Nổi Bật

Auto-trace

Không cần config — mọi `@weave.op()` tự được trace với input/output/latency

Dataset versioning

Khi dataset thay đổi → tự động tạo version mới. Track eval evolution theo thời gian

Model comparison

Số sánh GPT-4o vs Llama-3 vs Claude trên cùng evaluation suite side-by-side

Cost tracking

Tính tổng token cost per evaluation run. Budget-aware development

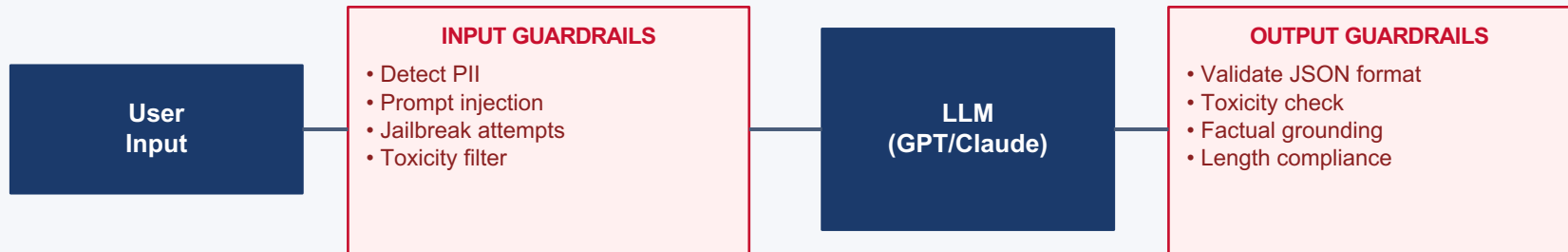
Weave — So Sánh Models Trên Cùng Eval Suite

Model	Faithfulness	Relevance	Latency P95	Cost/1M tokens	Tổng điểm
GPT-5-4	0.91	0.88	3.2s	\$2.5	★★★★★
GPT-5.4-mini	0.82	0.80	1.1s	\$0.75	★★★★
Claude 4.6 Sonnet	0.89	0.86	2.8s	\$3	★★★★★
Llama-3.1 70B	0.78	0.75	4.5s	\$0.05	★★★
Gemini 3.1 Pro	0.80	0.79	1.8s	\$2	★★★★

PHẦN 06

Guardrails & Safety Monitoring

Defense in Depth cho LLM Applications



Tools & Libraries

Guardrails AI

Python library. Validate JSON, detect PII, check toxicity, auto-reask khi fail.

Llama Guard 2

Meta open-source safety model. Classify input/output theo 14 harm categories.

NeMo Guardrails

NVIDIA. Dialog safety rails cho conversational AI với topical & safety rails.

Azure Content Safety

Microsoft managed service. Text + image moderation, customizable thresholds.

```
from guardrails import Guard
from guardrails.hub import ValidJson, DetectPII, ToxicLanguage

# Build guard pipeline
guard = Guard().use_many(
    ValidJson(on_fail="reask"),          # If not JSON → reask LLM
    DetectPII(
        pii_entities=["EMAIL", "PHONE", "SSN"],
        on_fail="fix"                  # Redact PII automatically
    ),
    ToxicLanguage(
        threshold=0.7,
        on_fail="noop"                 # Log but don't block
    )
)

# Wrap LLM call
result = guard(
    llm_api=openai.chat.completions.create,
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Summarize: {document}" }],
    num_reasks=2                       # Retry up to 2 times
)

# Access validated output
```

Alert Rules

IF: Toxicity rate > 0.5%

→ Slack alert → manual review

IF: Hallucination rate > 5%

→ Page on-call → investigation

IF: PII leak detected

→ Immediate block + audit log

IF: reask count > 3/hour

→ Review prompt quality

IF: Guard failure rate > 2%

→ Check guard configuration

PHẦN 07

Live Demo & Labs

LangSmith · Prompt A/B Test · RAGAS · Guardrails AI

Demo 1	RAG Pipeline + LangSmith Tracing Build RAG pipeline → Instrument với @traceable → View trace tree trong LangSmith → Identify bottleneck step → Filter slow traces	10 phút
Demo 2	Prompt A/B Test với Prompt Hub Push 2 prompt versions lên Hub → Implement 50/50 traffic router → Tag traces với version → So sánh metrics sau 100 queries	10 phút
Demo 3	RAGAS Evaluation Report Tạo 50 QA dataset → Run RAGAS với cả 2 prompt versions → So sánh faithfulness/relevance → Identify winner	8 phút
Demo 4	Guardrails AI — Block PII + Reformat Inject prompt với PII (email, phone) → Guardrails detect + redact → Inject bad JSON prompt → Guard reasks → Log incident	8 phút
Dashboard	LangSmith Dashboard Tour Latency heatmap per time window · Cost per day trend · Error rate by type · Token usage breakdown · Custom A/B tags	4 phút

Task 1: LangSmith Setup + RAG Instrumentation

1. Install: `pip install langsmith langchain-openai`
2. Set `LANGCHAIN_TRACING_V2=true` và `LANGCHAIN_API_KEY`
3. Build simple RAG pipeline với retriever + LLM
4. Thêm `@traceable` decorator cho custom functions
5. Chạy 100+ queries, verify traces xuất hiện trong dashboard

Task 2: Prompt Versioning + A/B Test

1. Push 2 system prompt versions lên LangSmith Prompt Hub
2. Version 1: concise (max 100 words), Version 2: detailed
3. Implement traffic router dựa trên `request_id % 2`
4. Tag mỗi trace với `prompt_version` metadata
5. So sánh avg latency, cost, output length giữa 2 versions

Task 3: RAGAS Evaluation

1. Tạo 50 QA pairs từ domain documentation
2. `pip install ragas` và setup evaluation dataset
3. Chạy `faithfulness` + `answer_relevancy` + `context_precision`
4. So sánh scores giữa 2 prompt versions
5. Document findings: version nào tốt hơn và tại sao

Task 4: Guardrails AI Validator

1. `pip install guardrails-ai` && `guardrails hub install hub://guardrails/detect_pii`
2. Implement Guard với `ValidJson` + `DetectPII`
3. Test với input chứa email/phone → verify redaction
4. Test với prompt yêu cầu JSON output → verify reask
5. Log incidents vào file để audit trail

Key Takeaways

Những điều quan trọng nhất từ buổi học hôm nay

1

Prompt là Code

Phải version control, review, test như code — không phải 'just text'. Mỗi thay đổi cần commit message, mỗi production deploy cần pin exact version.

2

LLM-as-Judge là Tốt, Nhưng Phải Calibrate

Scalable và rẻ hơn human eval 10-50x. Nhưng có bias — phải so sánh với human judgments định kỳ. Đo Cohen's kappa để track agreement.

3

Defense in Depth cho LLM Safety

Guardrails ở cả input và output layer. Không rely vào 1 điểm kiểm soát duy nhất. Mỗi layer bắt được loại lỗi khác nhau.

Tiếp Theo & Bài Tập Về Nhà

Ngày 23 Preview

Monitoring & Observability Stack

"Grafana dashboards, Prometheus metrics, alerting cho AI systems — biết model đang fail trước khi user phản hồi."

Topics:

- OpenTelemetry instrumentation
- Prometheus metrics cho LLM
- Grafana dashboard cho AI systems
- Alert rules & incident response

Bài Tập Về Nhà

1. Hoàn thành Lab #22

LangSmith + Prompt Versioning — nộp screenshot traces + RAGAS report

2. Cài Docker Compose

Setup Prometheus + Grafana cho pre-lab Ngày 23 (docker-compose.yml trên LMS)

3. Đọc trước

OpenTelemetry Python instrumentation guide — link trên LMS

4. Bonus

Thử so sánh 2 embedding models trên RAGAS context precision metric